

WEB DEV-II Question Bank – Detailed Answers

Prepared for Engineering Students

Section A – Detailed Answers

Q1. Define the DOM and explain its role in web development.

The Document Object Model (DOM) is a programming interface for HTML and XML documents. It represents a webpage as a tree-like structure where each HTML element is treated as an object. Using JavaScript, developers can access, modify, add, or delete elements dynamically. The DOM plays an important role in web development because it allows interactive and dynamic webpages. For example, when a button is clicked and text changes instantly without reloading the page, this is achieved using the DOM.

Q2. Explain DOM vs HTML.

HTML is the markup language used to create the structure of a webpage. It defines elements like headings, paragraphs, images, and buttons. The DOM, on the other hand, is the browser's internal representation of the HTML document. HTML is static content written by the developer, while the DOM is dynamic and can be manipulated using JavaScript. For example, an HTML paragraph can be changed through the DOM using JavaScript.

Q3. Difference between `textContent`, `innerHTML`, and `innerText`

1. `textContent`: Returns all text inside an element, including hidden text.
2. `innerText`: Returns only visible text and respects CSS styling.
3. `innerHTML`: Returns or sets the HTML content including tags.

Example:

```
element.textContent = 'Hello';
```

```
element.innerHTML = 'Hello';
```

Q4. Difference between `getElementById()`, `getElementsByClassName()`, `querySelector()`, and `querySelectorAll()`

`getElementById()` selects a single element using its ID.

`getElementsByClassName()` selects multiple elements having the same class.

`querySelector()` returns the first matching CSS selector.

`querySelectorAll()` returns all matching elements as a `NodeList`.

These methods are used to access HTML elements in JavaScript.

Q5. What are classList methods?

The classList property provides methods to manipulate CSS classes of HTML elements.

Methods include:

add() – adds a class

remove() – removes a class

toggle() – adds/removes a class depending on its existence

contains() – checks whether a class exists

Example: `element.classList.add('active');`

Q6. Discuss microtask and macrotask queue.

JavaScript uses queues for asynchronous execution.

Microtask Queue contains Promise callbacks and MutationObserver tasks.

Macrotask Queue contains `setTimeout()`, `setInterval()`, and I/O tasks.

The Event Loop first executes all microtasks before executing the next macrotask.

Q7. Difference between setTimeout() and setInterval()

`setTimeout()` executes a function only once after a specified delay.

`setInterval()` repeatedly executes a function after a specified interval.

Example:

`setTimeout(() => console.log('Hello'), 1000);`

`setInterval(() => console.log('Hi'), 2000);`

Q8. What is Callback Hell? Explain its disadvantages.

Callback Hell occurs when multiple nested callbacks make code difficult to read and maintain. It usually happens in asynchronous programming. Disadvantages include poor readability, difficult debugging, and complicated error handling. Promises and `async/await` are modern solutions to avoid Callback Hell.

Q9. What are Promise states in JavaScript?

A Promise represents the result of an asynchronous operation. The three states are:

1. Pending – operation is not completed.
2. Fulfilled – operation completed successfully.
3. Rejected – operation failed.

Promises use `.then()`, `.catch()`, and `.finally()` methods.

Q10. Explain the Event Loop.

The Event Loop is a mechanism that allows JavaScript to perform non-blocking asynchronous operations. It continuously checks the call stack and task queues. When the call stack becomes empty, tasks from microtask and macrotask queues are executed. This enables JavaScript to handle asynchronous code efficiently.

Q11. What is JSX in React?

JSX stands for JavaScript XML. It is a syntax extension that allows developers to write HTML-like code inside JavaScript. React converts JSX into JavaScript using Babel. Example:

```
const element = <Hello React/>;
```

Q12. What is one-way data binding?

In React, data flows only from parent components to child components through props. This is called one-way data binding. It makes applications predictable and easier to debug.

Q13. What are hooks? State the rules of hooks.

Hooks are special React functions that allow functional components to use state and lifecycle features.

Examples include `useState` and `useEffect`.

Rules of Hooks:

1. Hooks should only be called at the top level.
2. Hooks should only be called inside React functions.

Q14. Demonstrate the role of `useState`.

`useState` is a React Hook used for managing state in functional components.

Example:

```
const [count, setCount] = useState(0);
```

Here, `count` stores the state value and `setCount` updates it.

Q15. Define props in React.

Props are short for properties. They are used to pass data from parent components to child components.

Example:

```
function Student(props){ return {props.name};}
```

Q16. Differentiate between Local Storage and Session Storage.

Local Storage stores data permanently until manually removed.

Session Storage stores data only for the current browser session.

Local Storage has no expiration, while Session Storage is cleared when the browser tab is closed.

Q17. Explain the role of Babel.

Babel is a JavaScript compiler that converts modern JavaScript and JSX into browser-compatible JavaScript. It ensures compatibility with older browsers.

Q18. How does React manage frequent UI updates efficiently?

React uses the Virtual DOM to efficiently update the UI. It compares the Virtual DOM with the Real DOM using a process called reconciliation and updates only the changed elements.

Q19. Explain the dependency array in useEffect.

The dependency array controls when the useEffect hook executes.

If empty (`[]`), the effect runs only once.

If dependencies are provided, the effect runs whenever those values change.

Q20. Disadvantage of props drilling.

Props drilling occurs when data is passed through multiple intermediate components unnecessarily. It makes code difficult to manage and maintain. React Context API is used to solve this problem.

Section B – Long Answers

Q1. Event handling in JavaScript

Event handling allows JavaScript to respond to user actions such as clicks, keyboard input, and mouse movements. The `addEventListener()` method is commonly used.

Example:

```
button.addEventListener('click', function(){ alert('Button clicked'); });
```

Common events include click, mouseover, keydown, and submit. Event handling improves interactivity in webpages.

Q2. Event propagation

Event propagation determines how events travel in the DOM.

1. Bubbling: Event moves from child to parent.
2. Capturing: Event moves from parent to child.

Event listeners can use a third parameter to control capturing.

Example:

```
element.addEventListener('click', handler, true);
```

Q3. Execution Context in JavaScript

Execution Context is the environment in which JavaScript code runs.

Global Execution Context (GEC) is created when the program starts.

Function Execution Context (FEC) is created whenever a function is called.

The Call Stack manages execution order of contexts.

Q4. Lifecycle of a Promise

Promises solve the problem of Callback Hell.

A Promise has three states: Pending, Fulfilled, and Rejected.

Consumers include `.then()`, `.catch()`, and `.finally()`.

Example:

```
let promise = new Promise((resolve, reject) => {  
  resolve('Success');  
});  
promise.then(result => console.log(result));
```

Q5. Async/Await with error handling

async/await simplifies asynchronous programming.

try...catch is used for handling errors.

Example:

```
async function fetchData(){
  try{
    let response = await fetch(url);
  } catch(error){
    console.log(error);
  }
}
```

Q6. React Fetch API Example

React can fetch API data using useEffect.

Example:

```
useEffect(() => {
  fetch('https://jsonplaceholder.typicode.com/users')
    .then(res => res.json())
    .then(data => setUsers(data));
}, []);
```

This displays username and email of users.

Q7. Component lifecycle in React

React components go through mounting, updating, and unmounting phases. In functional components, useEffect is used to simulate lifecycle methods. For example, useEffect with an empty dependency array acts like componentDidMount.

Q8. Counter with popup using useEffect

useState manages counter state while useEffect controls side effects.

One button shows a popup and another updates the counter. useEffect executes whenever the counter changes.

Q9. User Information Form

A React form collects user data using controlled components.

useState is used to store form values. On submission, the entered data is displayed on the screen.

Q10. Counter Application

A counter application can be created using `useState`.

Functions `increment`, `decrement`, and `reset` modify the counter state dynamically.

Q11. Context API

Context API allows data sharing across components without passing props manually.

It solves props drilling by providing global state management.

Q12. Routing in React

React Router is used for navigation between pages.

Components like `BrowserRouter`, `Routes`, and `Route` are used to create multiple pages.

Outcome Based Question

The output order will be:

Start

Inside function

End

After await

Timeout

Reason:

1. 'Start' is printed first because it is synchronous.
2. The async function executes and prints 'Inside function'.
3. `await Promise.resolve()` pauses execution and moves the remaining code to the microtask queue.
4. 'End' is printed because synchronous code continues.
5. Microtasks execute before macrotasks, so 'After await' is printed next.
6. Finally, `setTimeout()` callback executes from the macrotask queue and prints 'Timeout'.